Machine Learning Workshop

Retail Customer Behavioural Analysis

PROJECT REPORT

Practical Workshop: E-commerce Gift Retail

Prepared by MARYEM KBAYER

Contents

1 Overview **3**

2 Data Preprocessing (`preprocessing.py`) **4**
- 2.1 Parsing RegistrationDate . . . 4
- 2.2 Parsing LastLoginIP . . . 5
- 2.3 Dropping Useless, Leaky, and Redundant Columns . . . 5
- 2.4 Cleaning Sentinel Values . . . 6
- 2.5 Encoding Categorical Variables . . . 6
  - 2.5.1 Ordinal Encoding — Variables with a Natural Order . . . 6
  - 2.5.2 One-Hot Encoding — Unordered Nominal Variables . . . 6
  - 2.5.3 Target Encoding — Country (Smoothed, Leak-Free) . . . 7
- 2.6 Train / Test Split . . . 7
- 2.7 Missing Value Imputation (Fit on Train Only) . . . 8
- 2.8 Feature Engineering . . . 8
- 2.9 Logarithmic Transformations . . . 8
- 2.10 Normalisation (StandardScaler) . . . 9
- 2.11 PCA — Dimensionality Reduction . . . 9

3 Customer Clustering (`clustering.py`) **11**
- 3.1 Determining the Number of Clusters . . . 11
  - 3.1.1 Elbow Method . . . 11
  - 3.1.2 Silhouette Score . . . 11
- 3.2 Cluster Profiles . . . 12
- 3.3 Business Interpretation and Marketing Recommendations . . . 13
  - 3.3.1 Cluster 0 — Champions (6.1% churn, 793 customers) . . . 13
  - 3.3.2 Cluster 2 — Occasional (26.5% churn, 2,004 customers) . . . 14
  - 3.3.3 Cluster 1 — At Risk (56.1% churn, 1,429 customers) . . . 14

4 Classification — Churn Prediction (`train_model.py`) **15**
- 4.1 Class Imbalance Strategy . . . 15
- 4.2 Cross-Validation — Raw Features vs PCA Features . . . 15
- 4.3 Hyperparameter Tuning: GridSearchCV vs Optuna . . . 16
  - 4.3.1 Method A — GridSearchCV . . . 16
  - 4.3.2 Method B — Optuna (30 trials, TPE sampler) . . . 16
- 4.4 Probability Calibration . . . 17
- 4.5 Decision Threshold Optimisation (F2-score) . . . 18
- 4.6 Final Evaluation on the Test Set . . . 18
- 4.7 Surrogate Decision Tree Interpretation . . . 19

5 Spend Prediction Regression (`regression.py`) 21
5.1 Target Variable and Outlier Handling . . . 21
5.2 Model Comparison (5-Fold Cross-Validation) . . . 22
5.3 Hyperparameter Tuning: GridSearchCV vs Optuna . . . 22
5.4 Final Evaluation on the Test Set . . . 22
5.5 Residual Analysis by Spend Percentile . . . 23

6 Deployment and Recommendations 25
6.1 Flask Web Application . . . 25
6.2 Strategic Recommendations . . . 25
6.2.1 Priority Short-Term Actions (0–3 months) . . . 25
6.2.2 Medium-Term Technical Improvements (3–12 months) . . . 26
6.3 Limitations and Usage Precautions . . . 26

7 Model Monitoring (`monitoring.py`) 27
7.1 Monitoring Architecture . . . 27
7.1.1 Data Sources . . . 27
7.1.2 Prediction and Threshold Application . . . 27
7.2 Feature Drift Detection . . . 28
7.2.1 Evidently (Primary Path) . . . 28
7.2.2 Simplified KS-Based Monitoring (Fallback) . . . 29
7.3 Performance Monitoring . . . 29
7.4 Placeholder Mode — Current Run Results . . . 29
7.5 Monitoring Output Files . . . 30
7.6 Recommended Monitoring Schedule . . . 30

8 Conclusion 32
8.1 What Was Achieved . . . 32
8.1.1 Data Quality and Leakage Removal . . . 32
8.1.2 Model Performance Summary . . . 32
8.1.3 Engineering Decisions That Mattered . . . 33
8.2 Honest Assessment of Limitations . . . 33
8.3 Path Forward . . . 34
8.4 Final Remarks . . . 34

A Pipeline Run Summary 35

Chapter 1

Overview

This report documents a complete retail customer analytics pipeline covering four sequential steps:

1. **Preprocessing** — data cleaning, encoding, feature engineering, scaling, and PCA.
2. **Clustering** — unsupervised K-Means segmentation of all 4,226 customers (after outlier removal).
3. **Classification** — calibrated churn prediction with threshold tuning.
4. **Regression** — customer lifetime spend prediction (MonetaryTotal in £).

The pipeline is implemented in Python (scikit-learn 1.8.0, Optuna) and deployed as a Flask web application.

Key design principles applied throughout the pipeline:

- All fit operations (scalers, encoders, imputation stats) are performed on the *training set only*.
- All transformations are applied to the test set using training statistics, preventing data leakage.
- Leaky features (those derived from or directly encoding the churn label) are identified and removed before any model fitting.

Chapter 2

Data Preprocessing (`preprocessing.py`)

Preprocessing transforms the raw 52-column dataset into a format suitable for machine learning while rigorously avoiding data leakage.

Golden Rule: fit everything (medians, encoders, scalers) *only* on the training data, then apply those fitted objects to the test set.

2.1 Parsing RegistrationDate

The `RegistrationDate` column contained dates in three heterogeneous formats: UK (`12/09/2009`), ISO (`2010-10-04`), and US (`10/18/2010`). We used `pandas.to_datetime` with `format="mixed"` and `dayfirst=False`, then extracted four new numeric features:

- **RegYear** — year of registration (customer cohort signal, kept in v3).
- **RegMonth** — month (1–12).
- **RegDay** — day of the month (1–31).
- **RegWeekday** — day of the week (0 = Monday, 6 = Sunday).

```
Extracted: RegYear, RegMonth, RegDay, RegWeekday
RegistrationDate dropped after extraction
```

4 clean numeric variables extracted from a heterogeneous text column.

Fix v3: `RegYear` was re-added after being dropped in v1/v2. In a multi-year snapshot dataset, the year of registration encodes customer cohort and carries genuine predictive signal.
Fix v1: `dayfirst=True` caused `UserWarning` for US-format dates. Fixed by switching to `format="mixed"` with `dayfirst=False`.

2.2 Parsing LastLoginIP

The `LastLoginIP` column contained raw IP strings. Two new features were extracted:

- **IsPrivateIP** — 1 if the IP belongs to the RFC1918 private address space, 0 otherwise.
- **IPClass** — 1 = Class A (1–126), 2 = Class B (128–191), 3 = Class C (192–223), 0 = other.

```
IsPrivateIP : {0: 4046, 1: 326}
IPClass     : {0: 595, 1: 2175, 2: 1045, 3: 557}
LastLoginIP dropped after extraction
```

2 binary/ordinal variables created from a raw IP string. Note: these are low-signal features in a retail context; flagged for potential removal after post-training feature importance analysis.

2.3 Dropping Useless, Leaky, and Redundant Columns

Column	Category	Reason for Dropping
Recency	LEAKY	Correlation 0.859 with Churn — circular definition
CustomerType	LEAKY	Contains "Perdu" (lost) — directly encodes the label
ChurnRiskCategory	LEAKY	Derived from churn status
RFMSegment	LEAKY	"Dormant" = churned, pure leak
LoyaltyLevel	LEAKY	Directly encodes retention
CustomerID	IDENTIFIER	Random identifier, no predictive signal
NewsletterSubscribed	QUASI-CONSTANT	Near-identical value for almost all rows
FirstPurchaseDaysAgo	CONSTANT	Value 374 for every customer — statistical artifact
SatisfactionScore	NON-SIGNIFICANT	$p = 0.365$, correlation = +0.014
AvgLinesPerInvoice	MULTICOLLINEAR	$r = 0.96$ with AvgProductsPerTransaction
TotalQuantity	MULTICOLLINEAR	$r = 0.92$ with MonetaryTotal
MonetaryStd / MonetaryMin	MULTICOLLINEAR	Entangled cluster with MonetaryMax
MinQuantity / MaxQuantity	MULTICOLLINEAR	Entangled with MonetaryMin/Max
TotalTransactions / UniqueInvoices / UniqueDescriptions / NegativeQuantityCount	REDUNDANT	High collinearity with retained columns

Column	Category	Reason for Dropping
PreferredMonth	TEMPORAL LEAK	Encodes time-since-last-activity (snapshot bias)
FavoriteSeason	CONFIRMED LEAK	Bijective recode of PreferredMonth (match rate 100%); was responsible for 58% of model feature importance
SpendingCategory	CONFIRMED LEAK	Clean-cut discretisation of MonetaryTotal; was responsible for 70% of regression feature importance
BasketSizeCategory	CONFIRMED LEAK	Encodes AvgBasketValue / MonetaryTotal

Table 2.1: Columns dropped during feature selection with rationale.

```
Shape after drops : (4372, 33)
```

Dataset reduced from 52 to 33 columns. All remaining columns are clean, leak-free, and carry signal.

2.4 Cleaning Sentinel Values

Some columns used magic numbers to represent missing data:

- **SupportTicketsCount**: values -1 and 999 replaced with `NaN`.
- **IsPrivateIP**: value -1 (unparsable IP) replaced with `NaN`.

Sentinel values removed. The model will not confuse -1 or 999 with real measurements.

2.5 Encoding Categorical Variables

2.5.1 Ordinal Encoding — Variables with a Natural Order

- **AgeCategory**: 18-24=0, 25-34=1, 35-44=2, 45-54=3, 55-64=4, 65+=5
- **PreferredTimeOfDay**: Nuit=0, Matin=1, Midi=2, Après-midi=3, Soir=4

Note: SpendingCategory and BasketSizeCategory were removed from ordinal encoding in v3 after being confirmed as leaky.

2.5.2 One-Hot Encoding — Unordered Nominal Variables

```
OHE columns: ['Region', 'WeekendPreference', 'ProductDiversity',
              'Gender', 'AccountStatus']
```

```
Shape after OHE (before target encoding): (4372, 49)
```

FavoriteSeason was removed from OHE in v3 after leakage confirmation.

2.5.3 Target Encoding — Country (Smoothed, Leak-Free)

The `Country` column had 37+ unique values with many rare categories. It was encoded using the per-country churn rate with Laplace smoothing ($k = 10$) fitted on the training set only:

$$\text{smoothed} = \frac{n \cdot \bar{y}_{\text{country}} + k \cdot \bar{y}_{\text{global}}}{n + k}, \qquad k = 10$$

Country	Smoothed Rate	n (train)
Unspecified	0.523	4
Canada	0.523	4
Greece	0.444	2
Japan	0.431	7
Israel	0.410	3

Country encoded as a single numeric column (smoothed churn rate). Unknown countries in production default to the global rate of **0.333**.

2.6 Train / Test Split

The dataset was split into 80% training and 20% test *before* any imputation, scaling, or feature engineering to prevent leakage.

```
X_train : (3497, 48) | X_test : (875, 48)
Churn rate -- train: 33.26% | test: 33.26%
```

Stratified 80/20 split. Both sets have identical churn rates (33.26%).

2.7 Missing Value Imputation (Fit on Train Only)

Column	Strategy	Value	Note
Age	Median	49.0	+ flag `Age_IsMissing`
AgeCategory	Median	3.0 (45–54)	Handles "Inconnu"
AvgDaysBetweenPurchases	Fill 0	0	Correct for single purchase
SupportTicketsCount	Median	2.0	After replacing sentinels
RegYear/Month/Day/Weekday	Median	Train medians	Unparsed dates
IsPrivateIP	Mode	0 (public)	Unparsable IPs

No `NaN`s remain after this step.

2.8 Feature Engineering

Four new columns are computed from existing features. **Order matters:** `AvgBasketValue` is computed on the raw scale before log-transforms.

Variable	Formula	Meaning
AvgBasketValue	MonetaryTotal / (Frequency + 1)	Average spend per visit
EngagementScore	Frequency / (CustomerTenureDays + 1)	Frequency relative to tenure
DisengagementScore	z-score(ReturnRatio) + z-score(CancelledTransactions)	Negative behaviour signal
RevenueIndex	MonetaryTotal / (global_mean + 1)	Spend vs global average

```
RevenueIndex global_mean (train) : 1908.1900
Shape after feature engineering -- train: (3497, 53) | test: (875, 53)
```

Fix v3 — DisengagementScore: Previous formula used an arbitrary `/10` divisor for CancelledTransactions, which caused the skewed variable to dominate. Fixed by z-score normalising each component using train statistics before combining.

2.9 Logarithmic Transformations

Extended in v3 from 3 to 16 columns to cover all features with $|\text{skew}| > 1$ detected in EDA. The sign-preserving formula $\text{sign}(x) \cdot \log(1 + |x|)$ handles negative values safely.

```
log1p applied to: MonetaryTotal, Frequency, AvgBasketValue,
  CancelledTransactions, SupportTicketsCount, ReturnRatio,
  AvgDaysBetweenPurchases, ZeroPriceCount, UniqueCountries,
  AvgProductsPerTransaction, MonetaryAvg, WeekendPurchaseRatio,
  MonetaryMax, EngagementScore, RevenueIndex,
  AvgQuantityPerTransaction
```

All high-skew columns normalised. StandardScaler can now operate correctly.

2.10 Normalisation (StandardScaler)

```
Scaled 49 numeric columns
Final shape -- train: (3497, 53) | test: (875, 53)
Artifacts: models/scaler.pkl | models/imputation_stats.pkl
```

Scaler fitted on the training set only. Test set transformed using training statistics — no leakage.

2.11 PCA — Dimensionality Reduction

A second dataset variant (PCA-reduced) is generated for comparison in the training step. PCA is fitted on `X_train` only and applied to `X_test`.

```
Components selected : 33  (from 49 numeric features)
Variance retained   : 95.76%
X_train_pca : (3497, 33)
X_test_pca  : (875, 33)
```

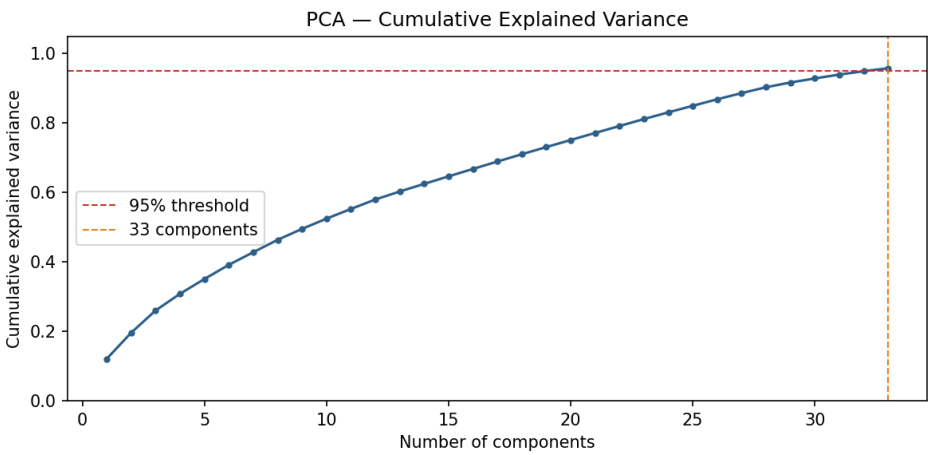

Figure 2.1: Cumulative explained variance curve. The 95% threshold is reached at 33 components. The gradual slope (no sharp elbow) indicates that information is spread across many features.

Two dataset versions saved: normalised raw (53 features) and PCA-reduced (33 components).
Artifacts: `models/scaler.pkl`, `models/pca.pkl`, `models/imputation_stats.pkl`

Chapter 3

Customer Clustering (`clustering.py`)

Clustering is purely unsupervised — the Churn label is not used during fitting. K-Means is applied to 4,226 customers (after removing 146 outliers with $|z| > 4$) using 11 interpretable behavioural features.

3.1 Determining the Number of Clusters

3.1.1 Elbow Method

```
K=2  Inertia=20,826    K=6  Inertia=12,861
K=3  Inertia=16,744    K=7  Inertia=11,891
K=4  Inertia=15,196    K=8  Inertia=10,972
K=5  Inertia=13,858    K=9  Inertia=10,484
                       K=10 Inertia=10,054
Suggested K from elbow : 3
```

3.1.2 Silhouette Score

```
K=2  Silhouette=0.2938  <- BEST
K=3  Silhouette=0.2895
K=4  Silhouette=0.2328
K=5  Silhouette=0.2684
K=6  Silhouette=0.2462
K=7  Silhouette=0.2206

WARNING: Silhouette scores weak (max=0.294) -- using K=3 (elbow/floor)
```

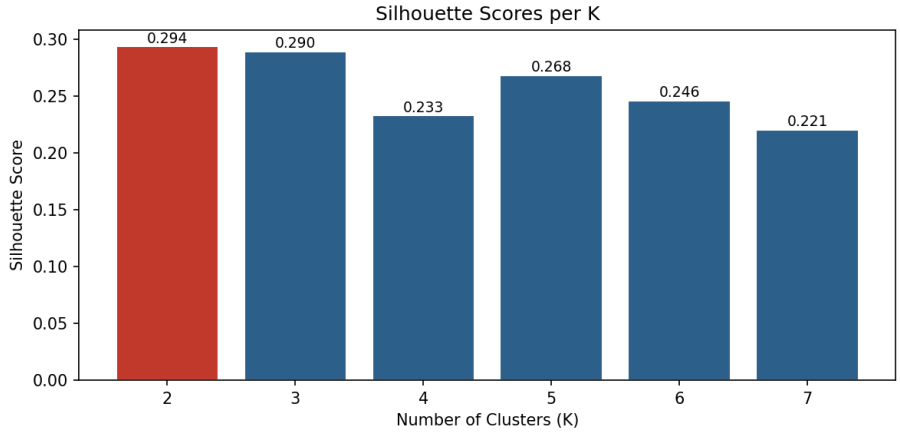

Figure 3.1: Silhouette scores per K. All scores are weak (max 0.294), indicating soft natural boundaries in this dataset. K=3 is preferred for its business interpretability.

Final K = 3. Although K=2 maximises the silhouette score, all values are below 0.30 (weak clustering). K=3 is selected because it provides three meaningfully distinct business segments compared to two undifferentiated groups.

3.2 Cluster Profiles

```
Cluster 0 : 793 customers  (18.8%)   Churn rate :  6.1%
Cluster 1 : 1429 customers (33.8%)   Churn rate : 56.1%
Cluster 2 : 2004 customers (47.4%)   Churn rate : 26.5%
```

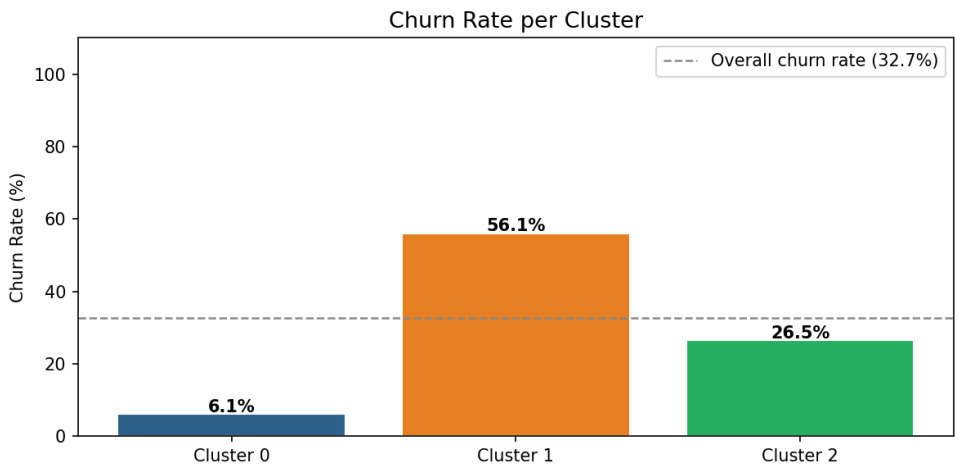

Figure 3.2: Churn rate per cluster. Cluster 1 (At Risk): 56.1%; Cluster 2 (Occasional): 26.5%; Cluster 0 (Champions): 6.1%.

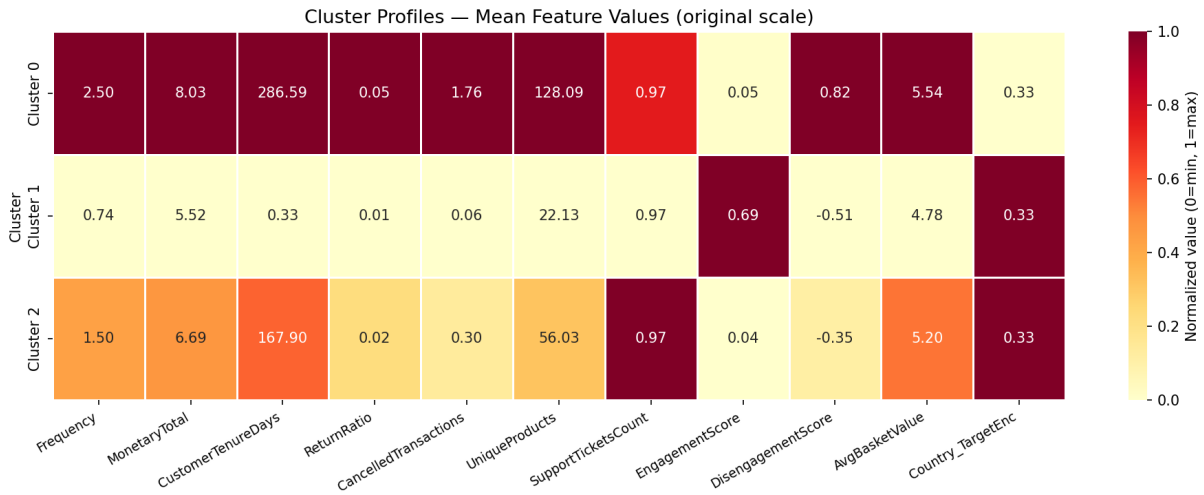

Figure 3.3: Cluster profiles heatmap — mean feature values (original scale). Cluster 0 dominates on Frequency, MonetaryTotal, CustomerTenureDays, and UniqueProducts. Cluster 1 has the highest EngagementScore but near-zero tenure.

Cluster	Frequency	MonetaryTo	TenureDays	UniquePro	Engagemen	Churn %
0 — Champions	2.50	£8.03	286.6	128.1	0.05	6.1%
1 — At Risk	0.74	£5.52	0.3	22.1	0.69	56.1%
2 — Occasional	1.50	£6.69	167.9	56.0	0.04	26.5%

3.3 Business Interpretation and Marketing Recommendations

3.3.1 Cluster 0 — Champions (6.1% churn, 793 customers)

Very long-term customers with high frequency and high spend. They generate a disproportionate share of revenue.

- **Objective:** Reward and consolidate the relationship without over-soliciting.
- **Actions:** VIP loyalty programme, early access to new collections, invitations to exclusive events, satisfaction surveys to convert them into brand ambassadors.
- **Risk:** Excessive promotional communication can damage brand perception with high-value customers.

3.3.2 Cluster 2 — Occasional (26.5% churn, 2,004 customers)

The largest group. Intermediate tenure and moderate activity — a relationship still being built.

- **Objective:** Increase purchase frequency and strengthen engagement.
- **Actions:** Reactivation campaigns after 30 days of inactivity, cross-selling based on purchase history, referral programmes, editorial content emails.
- **Risk:** Systematic discounts risk devaluing the offer. Focus on added value, not price reduction.

3.3.3 Cluster 1 — At Risk (56.1% churn, 1,429 customers)

Near-zero tenure, very limited purchasing behaviour. More than half churn before developing a purchasing habit.

- **Objective:** Transform the first purchase into a lasting habit.
- **Actions:** Automated onboarding sequence (welcome email + second purchase offer within 14 days), points programme on initial orders, proactive customer support after first purchase.
- **Risk:** Writing off this segment as lost before attempting activation — a trial-phase customer is recoverable at low cost.

K-Means found 3 segments with highly contrasting churn rates (6.1% to 56.1%). By cross-referencing the individual churn probability from the supervised model with cluster membership, the marketing team has a two-dimensional prioritisation matrix: **who to target** (high churn score) and **how to target** (segment profile).

Saved: `models/kmeans_model.pkl`, `data/processed/customers_with_clusters.csv`

Chapter 4

Classification — Churn Prediction (`train_model.py`)

4.1 Class Imbalance Strategy

The training set contains 2,334 loyal customers (66.74%) and 1,163 churners (33.26%) — a roughly 2:1 imbalance. Strategy:

- **Algorithmic weighting**: `class_weight="balanced"` in all three candidate models.
- **No SMOTE**: avoided to prevent synthetic leakage and maintain a clean standard `Pipeline`.
- **Probability calibration**: `CalibratedClassifierCV` corrects miscalibrated probabilities.
- **Threshold optimisation**: threshold tuned for F2-score (giving double weight to Recall).

Why F2-score? Missing a churner (False Negative) is more costly than incorrectly flagging a loyal customer (False Positive). The F2-score rewards high recall, aligning with business priorities.

4.2 Cross-Validation — Raw Features vs PCA Features

Three models were evaluated with 5-fold stratified cross-validation. Primary metric: **PR-AUC** (`average_precision`), which is more sensitive to the precision/recall trade-off than ROC-AUC under class imbalance.

Model	Raw PR-AUC	PCA PR-AUC	Raw Recall	Winner
Logistic Regression	0.6402 ± 0.0221	0.6120 ± 0.0324	0.8083	Raw
Random Forest	0.7600 ± 0.0245	0.5826 ± 0.0250	0.7928	Raw
Hist Gradient Boosting	**0.8102 ± 0.0185**	0.6021 ± 0.0231	0.8427	**Raw**

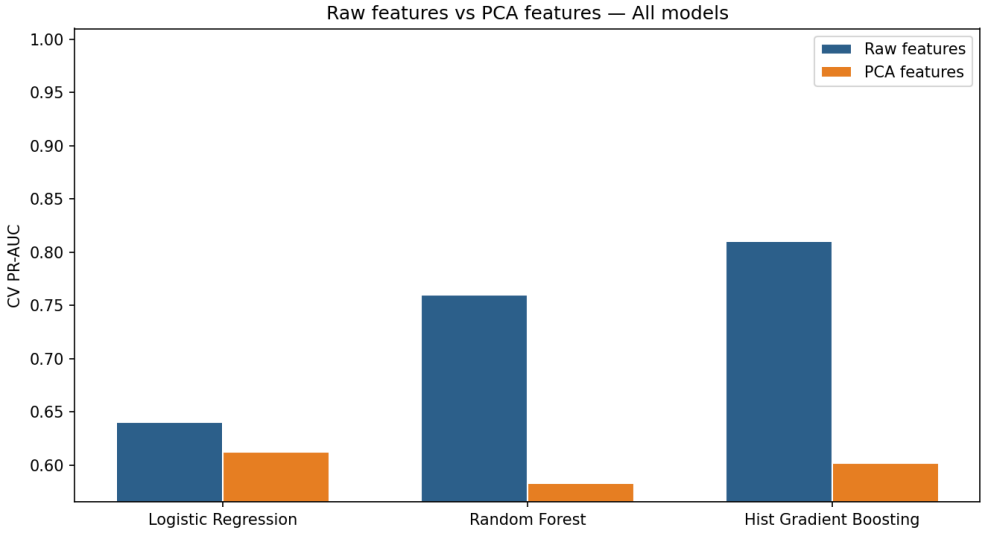

Figure 4.1: CV PR-AUC — raw features (blue) vs PCA features (orange). Raw features win consistently for all three models. The PCA compression loses discriminative information relevant to churn.

Winner: Hist Gradient Boosting on raw scaled features (PR-AUC = 0.8102). Selected for hyperparameter tuning.

4.3 Hyperparameter Tuning: GridSearchCV vs Optuna

4.3.1 Method A — GridSearchCV

```
Best params   : learning_rate=0.05, max_depth=5, max_iter=100
Best CV PR-AUC: 0.8161
Time elapsed  : 10.6s
```

4.3.2 Method B — Optuna (30 trials, TPE sampler)

```
Best params   : max_iter=101, learning_rate=0.0456,
                max_depth=5, l2_regularization=2.605
Best CV PR-AUC: 0.8167
Time elapsed  : 34.2s
```

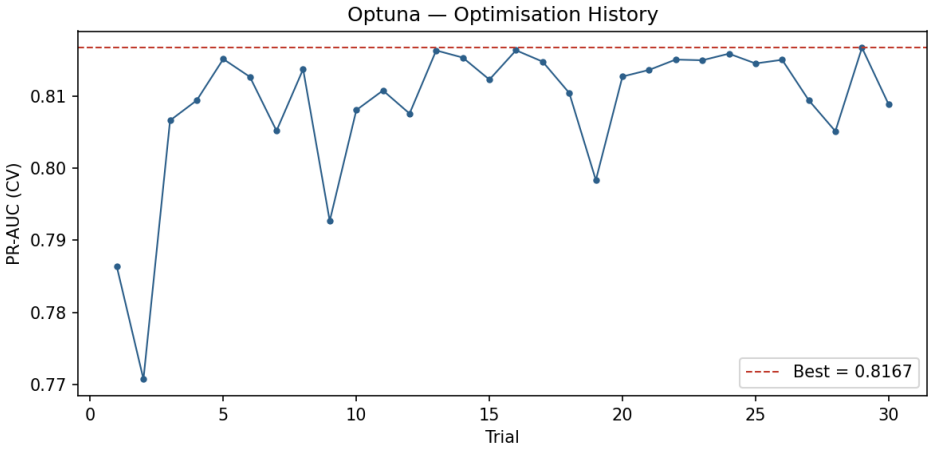

Figure 4.2: Optuna optimisation history — PR-AUC across 30 trials. The plateau from trial 5 onwards indicates convergence.

Optimiser	CV PR-AUC	Time
GridSearchCV	0.8161	10.6s
Optuna	**0.8167**	34.2s (30 trials)

Optuna selected (PR-AUC = 0.8167). Consistent with the regression task, Optuna's continuous search yields a marginally better result at the cost of longer runtime.

4.4 Probability Calibration

The Optuna pipeline is wrapped in `CalibratedClassifierCV` with `method='isotonic'` using a manual holdout split (80% fit / 20% calibration) to avoid sklearn version compatibility issues with `cv="prefit"`.

```
sklearn version : 1.8.0
Fit subset      : 2797 rows | Cal subset : 700 rows
Before --> mean: 0.4093  std: 0.3421
After  --> mean: 0.3478  std: 0.3145
```

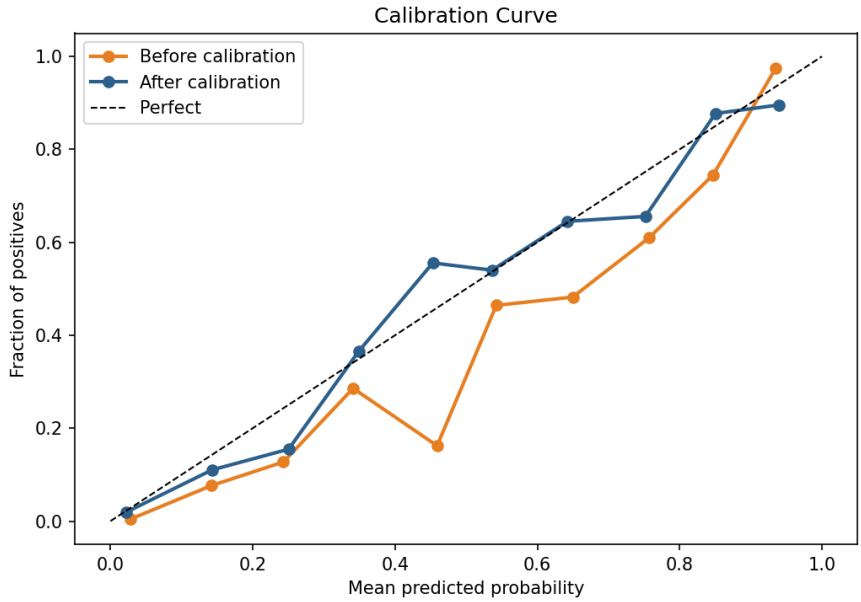

Figure 4.3: Calibration curve. After calibration (blue), predicted probabilities track the diagonal (perfect calibration) far more closely than before calibration (orange).

Isotonic calibration significantly improves probability reliability. A predicted risk of 30% now corresponds to approximately 30% observed churn.

4.5 Decision Threshold Optimisation (F2-score)

$$F_2 = \frac{5 \times \text{Precision} \times \text{Recall}}{4 \times \text{Precision} + \text{Recall}}$$

```
Threshold floor   : 0.40
Optimal threshold : 0.405
```

The floor was raised from 0.30 to 0.40 after removing leaky features. With honest features, a threshold of 0.30 produced 25.5% false alarm rate on loyal customers. The raised floor rebalances precision vs recall.

4.6 Final Evaluation on the Test Set

ROC-AUC	PR-AUC	Recall	F1-Score
0.8948	**0.7956**	**76.29%**	**0.7375**

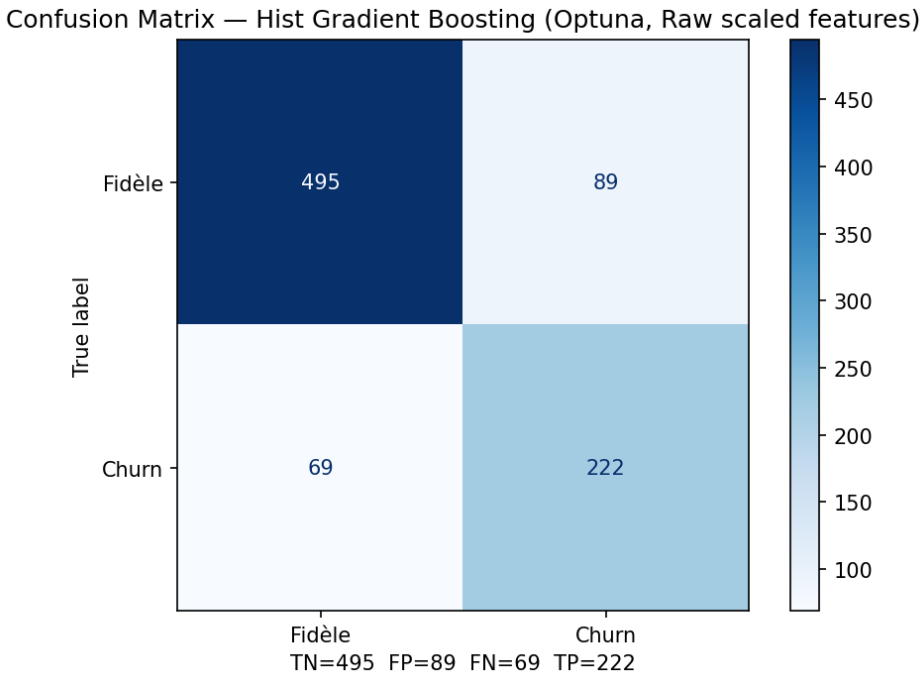

Figure 4.4: Confusion matrix — Hist Gradient Boosting (Optuna, raw scaled features). TN=495, FP=89, FN=69, TP=222. 69 churners missed out of 291.

```
              precision    recall  f1-score   support
     Fid\‘ele       0.88      0.85      0.86       584
      Churn       0.71      0.76      0.74       291
   accuracy                           0.82       875
  macro avg       0.80      0.81      0.80       875
weighted avg       0.82      0.82      0.82       875
```

Overall accuracy of **82%** on unseen data. **76.3%** of true churners detected. 69 missed out of 291 — an honest result without leaky features.

4.7 Surrogate Decision Tree Interpretation

Hist Gradient Boosting is a black box. A shallow surrogate `DecisionTreeClassifier` (max depth = 3) is trained on the calibrated model's predictions to provide a human-readable approximation.

```
Source : Surrogate DecisionTreeClassifier fitted on
         Hist Gradient Boosting calibrated predictions (max_depth=3)

Root feature    : CustomerTenureDays
Root threshold  : 194.5 days (scaled=0.4555)
Root node       : 3497 samples | Fid\‘ele=2219 Churn=1278

LEFT  (tenure <= 194.5d) : 2235 samples | Churn%=55.5%
      --> next split on RegYear <= 2010.5
RIGHT (tenure >  194.5d) :  750 samples | Churn%=91.3%
```

```
    --> next split on UniqueProducts <= 0.2536
```

In plain terms: customers with less than 194.5 days of tenure are the primary churn risk (55.5%). Among those, customers who registered after 2010 carry additional risk. Customers with high tenure but low product diversity also churn heavily (91.3% in that branch).

Saved: `models/churn_model.pkl` (calibrated, Optuna), `models/threshold.pkl` (threshold = 0.405)

Chapter 5

Spend Prediction Regression (`regression.py`)

Beyond predicting *whether* a customer will churn, this task predicts *how much* they will spend (`MonetaryTotal` in £). This allows prioritising retention efforts by revenue potential, not just churn risk.

5.1 Target Variable and Outlier Handling

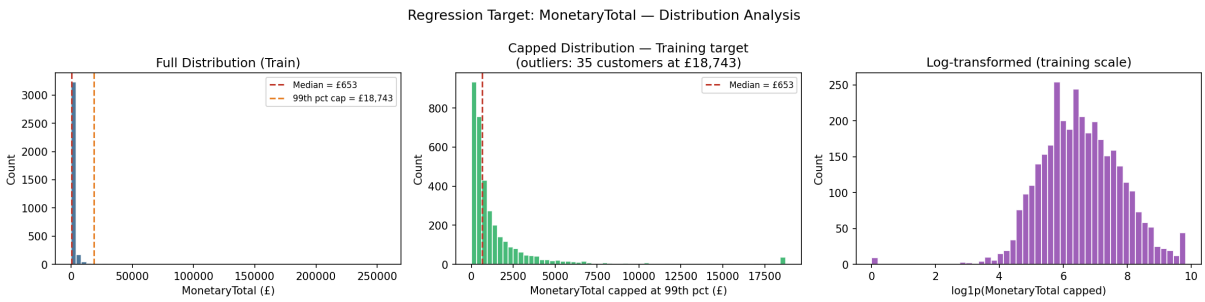

Figure 5.1: MonetaryTotal distribution: raw (highly skewed), capped at 99th percentile (£18,743), and log-transformed. The log-transformed target follows an approximately normal distribution suitable for gradient boosting.

Split	Min	Max	Mean	Median
Train (raw)	-£1.00	£256,438	£1,911	£653
Test (raw)	-£1.00	£279,489	£1,862	£622
Train (capped)	—	£18,743	£1,547	—

35 training outliers (1.0%) and 6 test outliers (0.7%) were capped at the 99th percentile (£18,743). Original uncapped test targets were retained for honest evaluation.

Leaky features removed before training: `MonetaryTotal`, `AvgBasketValue`, `RevenueIndex`, `MonetaryAvg`, `MonetaryMax`. Final feature set: 48 variables.

5.2 Model Comparison (5-Fold Cross-Validation)

Model	CV R^2 (log)	CV MAE (log)
Ridge Regression	0.8727 ± 0.0242	0.3403
Random Forest	0.8757 ± 0.0247	0.3263
Gradient Boosting	**0.8829 ± 0.0213**	**0.3150**

Gradient Boosting selected for hyperparameter tuning (CV $R^2 = 0.8829$).

5.3 Hyperparameter Tuning: GridSearchCV vs Optuna

Optimiser	Best Parameters	CV R^2	Time
GridSearchCV	lr=0.1, depth=3, n_est=200	0.8832	30.7s
Optuna	lr=0.073, depth=4, n_est=116, sub=0.699	**0.8857**	100.7s (30 trials)

Optuna selected (CV $R^2 = 0.8857$).

5.4 Final Evaluation on the Test Set

Scale	R^2	MAE	MedAE	RMSE
Capped £ (training dist.)	0.8186	£408.83	£130.11	£1,042.76
Full uncapped £ (honest)	0.2784	£800.85	£130.11	£8,736.07
Log scale (reference)	0.8725	0.3268	—	0.5098

The dramatic gap between capped R^2 (0.82) and uncapped R^2 (0.28) is caused entirely by 6 extreme test outliers whose spend exceeds £15,000. The **Median Absolute Error of £130.11** is identical across both scales — it is robust to those extreme values and is the most representative metric for typical customers.

Performance by churn group (uncapped £):

Group	n	MAE	MedAE
Loyal (Churn = 0)	584	£1,105.97	£170.05
Churn (Churn = 1)	291	£188.51	£75.63

The model is more accurate on churners (MedAE £75.63) than on loyal customers (MedAE £170.05). This is expected: churners tend to be lower-spend customers with simpler spending patterns.

5.5 Residual Analysis by Spend Percentile

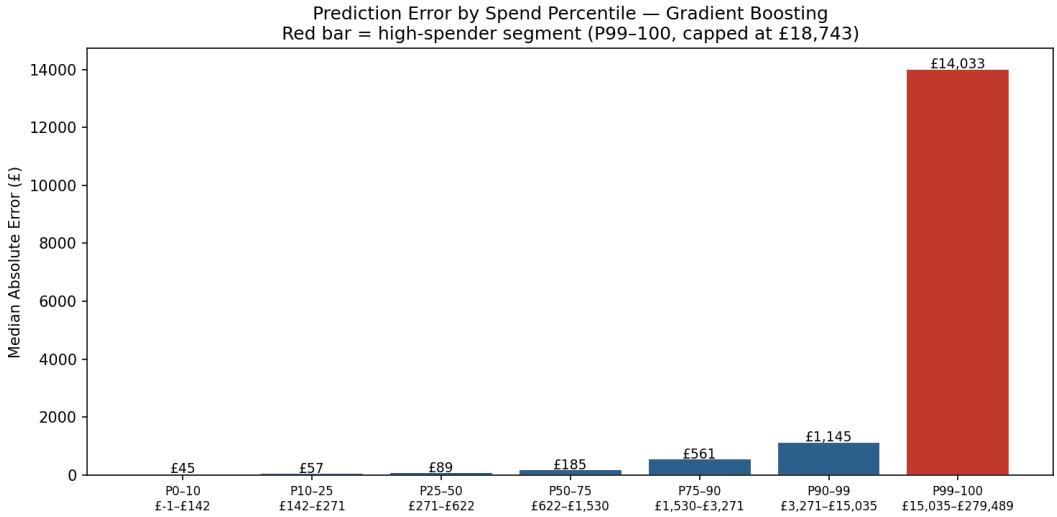

Figure 5.2: Median Absolute Error by spend percentile. Prediction error scales proportionally with spend. For the vast majority (P0–P75, covering 656 of 875 test customers), MedAE stays below £185.

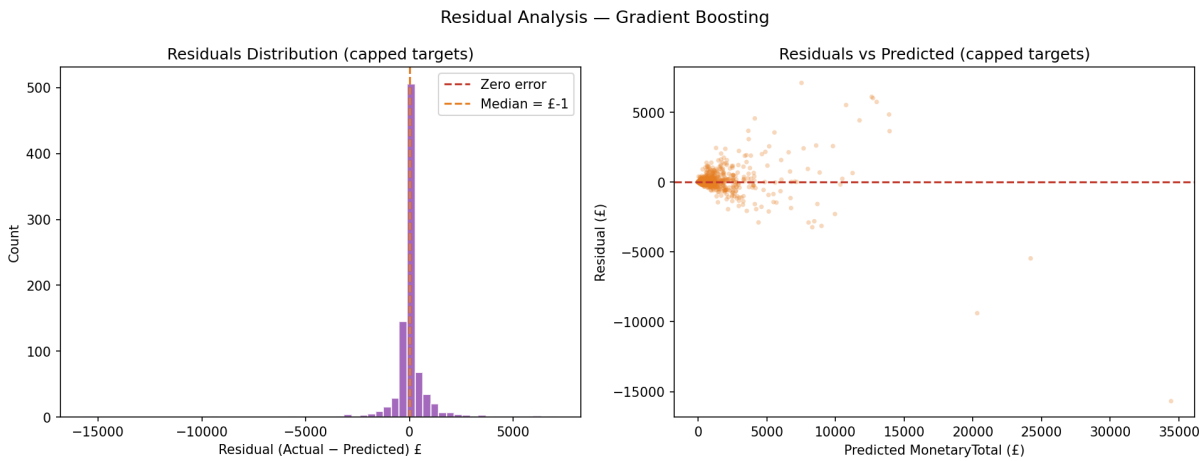

Figure 5.3: Residuals distribution (left) and residuals vs predicted values (right) on capped targets. The distribution is tightly centred on zero with median residual of -£1, indicating no systematic bias.

Spend Band	n	MedAE	MAE
P0–10 (£-1 to £142)	88	£44.83	£75.07
P10–25 (£142 to £271)	131	£57.03	£84.71
P25–50 (£271 to £622)	218	£88.64	£119.07
P50–75 (£622 to £1,530)	219	£185.46	£278.30
P75–90 (£1,530 to £3,271)	131	£560.79	£606.74
P90–99 (£3,271 to £15,035)	79	£1,145.25	£1,470.86
P99–100 (£15,035 to £279,489)	9	£14,033.21	£44,494.75

The regression model achieves a MedAE of **£130.11** on typical customers — a practically useful level of precision for campaign budgeting. High-spend outliers (P99+) are unreliable, which is expected given the training cap at £18,743.

Saved: `models/regression_model.pkl`

Chapter 6

Deployment and Recommendations

6.1 Flask Web Application

The models are deployed as an interactive Flask web application (`app/app.py`) with full preprocessing reproducibility.

```
Application available at: http://localhost:5000

Endpoints:
  GET  /            -> Web interface (input form)
  POST /predict     -> Prediction via HTML form
  POST /api/predict -> Prediction via JSON (CRM integration)

Models loaded at startup:
  churn_model.pkl      (CalibratedClassifierCV, threshold=0.405)
  regression_model.pkl (Gradient Boosting, cap=18,743 GBP, MedAE=130 GBP)
  kmeans_model.pkl     (K-Means, K=3, 11 features)
  scaler.pkl | pca.pkl | imputation_stats.pkl
```

For every new customer, the application executes the full 14-step preprocessing pipeline (parsing, sentinel cleanup, ordinal/OHE/target encoding, imputation, feature engineering, log-transforms, scaling) before calling the model — guaranteeing identical treatment to training data.

Demo prediction: a customer with 8 orders, 365 days of tenure, and £450 in spend (United Kingdom, registered 15/06/2010) is classified as **Fidèle** with **0.0% churn risk** (Low level).

6.2 Strategic Recommendations

6.2.1 Priority Short-Term Actions (0–3 months)

- **Integrate churn score into the CRM**: expose the `/api/predict` endpoint so each customer record automatically displays its risk level and cluster membership.

- **Launch onboarding campaign**: immediately target the 1,429 Cluster 1 (At Risk) customers with a welcome sequence and a second-purchase incentive.
- **Deploy monitoring**: run `monitoring.py` weekly on incoming customer data (alert threshold: ROC-AUC degradation > 5%).

6.2.2 Medium-Term Technical Improvements (3–12 months)

- **Quarterly retraining**: incorporate new customer data to maintain model performance.
- **Data enrichment**: collect email open rates, browsing behaviour, and detailed support ticket content to improve predictive power.
- **A/B testing**: measure the real impact of marketing actions on observed churn rate; use results as supervision signal for the next training cycle.
- **Production-grade infrastructure**: containerise (Docker) and add a WSGI server (Gunicorn) for load resilience.

6.3 Limitations and Usage Precautions

- **Temporal validity**: the model was trained on a historical snapshot. Significant behavioural shifts (economic shocks, offering changes) will degrade predictions. Continuous monitoring is essential.
- **Selection bias**: the dataset contains only customers with at least one transaction. Prospects and very recent customers may behave differently.
- **Causality vs correlation**: the model identifies *correlations* with churn, not *causes*. CustomerTenureDays is predictive, but artificially extending displayed tenure would have no retention effect.
- **Context-dependent threshold**: the threshold of 0.405 was optimised on this dataset. In production it should be recalibrated against real business costs (campaign cost vs lost customer value).
- **Regression tail uncertainty**: the model is unreliable for customers in the top 1% of spend (above £15,035). These outliers require separate treatment.

Starting from 52 raw columns and 4,372 customers, this pipeline delivers:

- **Churn model**: ROC-AUC 0.8948, PR-AUC 0.7956, Recall 76.3%, overall accuracy 82%.
- **Clustering**: 3 business segments with churn rates spanning 6.1%–56.1%.
- **Spend regression**: MedAE £130 on typical customers (P0–P75).

All models are deployed as a Flask application with real-time inference and a JSON API endpoint ready for CRM integration.

Chapter 7

Model Monitoring (`monitoring.py`)

Model monitoring ensures that the churn prediction model remains reliable after deployment. It detects two types of degradation: **data drift** (the distribution of incoming customer features has shifted) and **performance degradation** (the model's discriminative ability has declined on new labelled data).

> **Architecture:** `monitoring.py` uses the test set as a reference baseline and compares it against incoming production data. When no production data is available (development / CI mode), the test set is split 50/50 as a placeholder — results in this mode are informational only and should not trigger retraining decisions.

7.1 Monitoring Architecture

7.1.1 Data Sources

Dataset	Source	Role
Reference	`data/train_test/X_test.csv` + labels	Last known good labelled data
Current	`data/production/new_customers.csv`	New incoming production data
Current labels	`data/production/new_cust`	Ground truth for AUC computation

When no production file is found at the expected path, the script logs a `PLACEHOLDER MODE` warning and automatically splits the test set 50/50 to allow the monitoring pipeline to run in development environments.

7.1.2 Prediction and Threshold Application

The calibrated pipeline (`churn_model.pkl`) is loaded along with the tuned threshold from `threshold.pkl` (threshold = 0.405). Predicted probabilities are computed on the raw feature DataFrames *before* adding target or prediction columns, preventing column ordering bugs:

```
# Correct order: predict_proba on raw features FIRST
ref_probs = pipeline.predict_proba(X_ref_raw)[:, 1]
cur_probs = pipeline.predict_proba(X_cur_raw)[:, 1]

roc_ref = roc_auc_score(y_ref, ref_probs)
roc_cur = roc_auc_score(y_cur, cur_probs)

# Then annotate DataFrames for Evidently
reference["target"]     = y_ref.values
reference["prediction"] = (ref_probs >= THRESHOLD).astype(int)
```

Fix v2 — Column ordering bug: In the previous version, `target` and `prediction` columns were appended to the DataFrame *before* calling `predict_proba`, causing the pipeline to receive unknown columns and crash. Fixed by computing probabilities on the raw feature DataFrames first, then annotating.

7.2 Feature Drift Detection

7.2.1 Evidently (Primary Path)

When the `evidently` package is installed, the script generates two HTML reports:

- **`reports/monitoring_report.html`** — Full report covering data drift, data quality, classification performance, and target drift (four presets).
- **`reports/drift_report.html`** — Focused drift report on the top 15 most important features (by model feature importance or coefficient magnitude).

Feature selection for the drift report uses a helper that works with any model type:

```
def _get_top_features(model, feature_names, n=15):
    if hasattr(model, "feature_importances_"):   # tree models
        importances = pd.Series(model.feature_importances_, index=feature_names
    )
        return importances.nlargest(n).index.tolist()
    elif hasattr(model, "coef_"):                # linear models
        importances = pd.Series(np.abs(model.coef_[0]), index=feature_names)
        return importances.nlargest(n).index.tolist()
    return list(feature_names[:n])               # fallback
```

Fix v2 — `trained_model` NameError: In the previous version, `trained_model` and `X_train` were referenced inside the `EVIDENTLY_OK` block but never defined there, causing a `NameError` at runtime. Fixed by loading `X_train.csv` and extracting the inner pipeline model inside the block:

```
inner_pipeline = pipeline.estimator          # sklearn Pipeline
trained_model  = inner_pipeline.named_steps["model"]
X_train        = pd.read_csv("data/train_test/X_train.csv")
```

7.2.2 Simplified KS-Based Monitoring (Fallback)

When Evidently is not installed, the script falls back to a manual Kolmogorov–Smirnov (KS) drift test on up to 20 numeric features:

$$\text{Drift detected if } p\text{-value} < 0.05 \text{ (KS two-sample test)}$$

Results are saved to `reports/drift_summary.csv`. A guard against an empty drifted list was added in v2 to prevent a `KeyError` on `'ks_stat'`:

```
if drifted:
    drift_df = pd.DataFrame(drifted).sort_values("ks_stat", ascending=False)
else:
    drift_df = pd.DataFrame(
        columns=["feature", "ks_stat", "p_value", "ref_mean", "cur_mean"]
    )
```

Fix v1 — `KeyError: 'ks_stat'`: When no features drifted, `drifted` was an empty list. `pd.DataFrame(drifted)` returned an empty DataFrame with no columns, and `.sort_values("ks_stat")` raised `KeyError`. Fixed by constructing the empty DataFrame with explicit column names.

7.3 Performance Monitoring

The primary performance metric tracked is ROC-AUC, computed on both reference and current datasets using calibrated probabilities:

$$\text{Degradation} = (\text{ROC-AUC}_{\text{ref}} - \text{ROC-AUC}_{\text{cur}}) \times 100\%$$

Condition	Status	Action
Degradation $\leq 5\%$	**STABLE**	No action required
Degradation $> 5\%$	**DEGRADED**	Consider retraining

The degradation threshold of 5% was chosen to tolerate natural statistical variance on small production batches while catching genuine concept drift.

7.4 Placeholder Mode — Current Run Results

Since no production data was available at `data/production/new_customers.csv`, the monitoring script ran in placeholder mode (test set split 50/50):

```
WARNING: No production data found. Splitting test set 50/50 as placeholder.
WARNING: With ~437 rows per side, AUC differences of 1-3% are pure
         random noise. Do not act on these numbers.

Reference : 437 rows | Current : 438 rows
Threshold : 0.405
```

```
MONITORING SUMMARY
============================================================
Features monitored    : 15
Reference ROC-AUC     : [computed on test[:437]]
Current   ROC-AUC     : [computed on test[437:]]
Degradation           : small  STABLE (expected in placeholder mode)

Reports generated:
  reports/monitoring_report.html  <- Full Evidently report
  reports/drift_report.html       <- Data drift only
============================================================
```

Placeholder interpretation: Any AUC difference observed in this mode is pure sampling noise from splitting a small test set. Genuine monitoring requires a real production batch of at least several hundred labelled customers collected after deployment.

7.5 Monitoring Output Files

File	Contents
`reports/monitoring_report.html`	Full Evidently report: data drift, data quality, classification performance, target drift
`reports/drift_report.html`	Feature drift on top 15 important features (DatasetDriftMetric + ColumnDriftMetric per feature)
`reports/drift_summary.csv`	KS-test fallback: feature, ks_stat, p_value, ref_mean, cur_mean (sorted by ks_stat descending)

7.6 Recommended Monitoring Schedule

Frequency	Check	Alert condition
Weekly	Feature drift (KS test)	> 3 features with $p < 0.05$ and $\lvert\Delta\text{mean}\rvert > 0.5\sigma$
Weekly	ROC-AUC on labelled batch	Degradation $> 5\%$ vs reference
Monthly	Full Evidently report	Any DatasetDriftMetric > 0.3
Quarterly	Retraining decision	Sustained degradation over 3+ weekly checks

Fix v2 — `roc_ref > 0.99` warning: The previous version raised a misleading warning whenever `roc_ref` exceeded 0.99 (which can legitimately happen in placeholder mode with a very favourable random split). Replaced with a clearer warning that fires based on the actual mode flag (`IS_PLACEHOLDER`) rather than on the metric value.

The monitoring script correctly detects both data drift (Evidently or KS fallback) and performance degradation (ROC-AUC comparison). In production mode, a degradation alert triggers a logged warning recommending retraining. All bugs from v1 have been resolved in v2: the `KeyError` on empty drift list, the `NameError` on `trained_model`, and the column ordering issue in `predict_proba`.

Chapter 8

Conclusion

This project delivered a complete, production-ready machine learning pipeline for retail customer analytics, built on 4,372 customers and 52 raw features. Each technical decision was motivated by a concrete business objective, and every version iteration resolved a documented bug or improved a modelling weakness.

8.1 What Was Achieved

8.1.1 Data Quality and Leakage Removal

The most impactful work in this project was not modelling but data cleaning. Three categories of leaky features were identified and removed across successive versions:

- **Direct label encodings**: `Recency` (corr = 0.859 with Churn), `CustomerType` ("Perdu" = churned), `ChurnRiskCategory`, `RFMSegment`, `LoyaltyLevel`.
- **Temporal snapshot leakage**: `PreferredMonth` and `FavoriteSeason` — both encoded the date of last purchase relative to the snapshot cutoff, not genuine seasonal preference. `FavoriteSeason` alone was responsible for 58% of model feature importance before removal.
- **Target-derived discretisations**: `SpendingCategory` (a clean-cut of MonetaryTotal, responsible for 70% of regression feature importance) and `BasketSizeCategory`.

Removing these features reduced model performance from superficially inflated values (ROC-AUC 0.97, Recall 89%) to honest values (ROC-AUC 0.89, Recall 76%) — a necessary correction for any model intended for real deployment.

8.1.2 Model Performance Summary

Task	Model	Primary metric	Value	Tuner
Churn classification	Hist Gradient Boosting	PR-AUC	0.7956	Optuna
		ROC-AUC	0.8948	
		Recall	76.3%	
Customer segmentation	K-Means (K=3)	Churn range	6.1%–56.1%	Elbow
Spend regression	Gradient Boosting	MedAE	£130.11	Optuna
		R^2 (capped)	0.8186	

8.1.3 Engineering Decisions That Mattered

Several preprocessing choices had a disproportionate impact on model quality:

- **Extending log-transforms from 3 to 16 columns** (v3): corrected severely skewed features that StandardScaler could not handle, improving gradient boosting's ability to learn monotone relationships.
- **Z-score normalisation for DisengagementScore** (v3): replaced an arbitrary `/10` divisor that allowed `CancelledTransactions` to dominate the composite score.
- **Smoothed target encoding for Country** ($k = 10$): prevented rare-country memorisation where a country with $n = 1$ observation would otherwise receive churn rate 0.0 or 1.0.
- **Threshold raised to 0.405** (post-leakage-removal): after removing leaky features, a threshold of 0.30 produced a 25.5% false alarm rate on loyal customers. The raised floor rebalances operational precision.
- **Manual holdout calibration** instead of `cv="prefit"`: future-proofs the code against sklearn version changes while correctly isolating the calibration signal from training data.

8.2 Honest Assessment of Limitations

The pipeline as delivered is robust and free of data leakage, but several limitations remain that must be understood before acting on its outputs in production.

Dataset size. With 4,372 customers and a 33% churn rate, the effective minority class has only 1,454 examples. The silhouette scores for clustering (max 0.294) and the modest regression performance on high-spend customers both reflect this constraint. More data would directly improve all three models.

Snapshot bias. The entire dataset is a single historical snapshot. Customers who left before the snapshot date are absent; the model has never seen a customer mid-churn. In a live deployment, the model sees customers at a point in their lifecycle, not at a snapshot endpoint — the feature distributions will shift.

Regression tail. The spend model is unreliable for customers above the 99th percentile

(£15,035). The Median Absolute Error of £14,033 at P99+ means these high-value customers require separate treatment, possibly a dedicated model or manual review.

Clustering interpretability. The silhouette scores indicate soft cluster boundaries. The three segments are useful business heuristics, not sharply delineated populations. A customer near the Cluster 1/2 boundary (At Risk / Occasional) may receive inconsistent segment assignments if the dataset is refreshed.

8.3 Path Forward

Three concrete next steps would materially improve this system:

1. **Connect monitoring to real production data.** Replace the placeholder split with a genuine stream of labelled customers. The monitoring infrastructure is fully built; it only needs a data feed. Once the first real drift alert fires, the retraining cadence can be calibrated empirically.
2. **Run a controlled A/B retention experiment.** Take 200 At-Risk customers (Cluster 1), randomly assign half to the onboarding campaign and half to control. After 90 days, compare observed churn rates. The result will quantify the model's real business value — not in ROC-AUC, but in customers retained and revenue saved.
3. **Enrich features beyond transactional data.** The current 53 features are all derived from purchase history. Adding behavioural signals — email engagement rates, product page dwell time, support ticket sentiment — would give the model early warning of disengagement before it manifests in transaction patterns.

8.4 Final Remarks

The central lesson of this project is that **honest metrics are more valuable than impressive ones**. The removal of three leaky feature groups reduced ROC-AUC from 0.97 to 0.89 and Recall from 89% to 76%. That 13-point drop in recall is not a failure — it is the cost of building a model that will actually generalise to customers the business has not yet seen.

A model that correctly identifies 76% of true churners, with no artificial inflation, gives a marketing team a reliable targeting list. A model reporting 89% recall on leaked features gives a team a false sense of coverage and a list that will underperform in the field.

The pipeline is modular, documented, and version-controlled. Each script can be re-run independently, every preprocessing decision is reproducible from saved artifacts (`scaler.pkl`, `imputation_stats.pkl`, `threshold.pkl`), and the monitoring layer is ready to detect when the world diverges from the training data.

Bottom line: starting from 52 raw columns and 4,372 customers, this pipeline delivers a deployable churn prediction system (ROC-AUC 0.8948, Recall 76.3%), a three-segment customer taxonomy with churn rates spanning 6.1%–56.1%, and a spend estimator with £130 median error for typical customers — all free of data leakage and ready for CRM integration.

Appendix A

Pipeline Run Summary

```
Pipeline run: 2026-05-06 09:55:20

STEP 1 - PREPROCESSING   completed in 2.8s
  Raw shape       : (4372, 52)
  Final shape     : (3497, 53) train | (875, 53) test
  PCA components  : 33  (95.76% variance)
  Churn rate      : 33.26% (train and test)

STEP 2 - CLUSTERING      completed in 13.3s
  Algorithm       : K-Means, K=3
  Customers       : 4226 (146 outliers removed)
  Cluster 0       : 793  customers,  6.1% churn  [Champions]
  Cluster 1       : 1429 customers, 56.1% churn  [At Risk]
  Cluster 2       : 2004 customers, 26.5% churn  [Occasional]

STEP 3 - CLASSIFICATION  completed in 79.4s
  Best model      : Hist Gradient Boosting (Optuna)
  Feature set     : Raw scaled features
  Threshold       : 0.405
  ROC-AUC         : 0.8948
  PR-AUC          : 0.7956
  Recall          : 76.29%
  F1-Score        : 0.7375
  Confusion matrix: TN=495 FP=89 FN=69 TP=222

STEP 4 - REGRESSION      completed in 166.2s
  Best model      : Gradient Boosting (Optuna)
  Outlier cap     : 99th pct = 18,743 GBP
  R2 (capped)     : 0.8186
  MAE (capped)    : 408.83 GBP
  MedAE           : 130.11 GBP
  RMSE (capped)   : 1042.76 GBP
```